

My Project

Generated by Doxygen 1.17.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 Studentas() [1/2]	8
4.1.2.2 Studentas() [2/2]	9
4.1.3 Member Function Documentation	9
4.1.3.1 addPaz()	9
4.1.3.2 clearPaz()	9
4.1.3.3 getEgz()	9
4.1.3.4 getMedrez()	9
4.1.3.5 getPaz()	9
4.1.3.6 getRez()	10
4.1.3.7 info()	10
4.1.3.8 operator=() [1/2]	10
4.1.3.9 operator=() [2/2]	10
4.1.3.10 refEgz()	10
4.1.3.11 refMedrez()	11
4.1.3.12 refPaz()	11
4.1.3.13 refRez()	11
4.1.3.14 setEgz()	11
4.1.3.15 skaiciuoti()	11
4.2 Zmogus Class Reference	12
4.2.1 Detailed Description	12
4.2.2 Member Function Documentation	12
4.2.2.1 getPavarde()	12
4.2.2.2 getVardas()	13
4.2.2.3 info()	13
4.2.2.4 refPavarde()	13
4.2.2.5 refVardas()	13
4.2.2.6 setPavarde()	13
4.2.2.7 setVardas()	13
4.2.3 Member Data Documentation	13
4.2.3.1 pavarde	13

4.2.3.2 vardas	13
5 File Documentation	15
5.1 funk.cpp	15
5.2 studentai.h	22
5.3 versija_v2.0.cpp	24
Index	27

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	12
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas		7
Zmogus		12

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

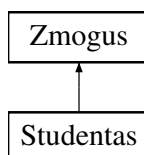
funk.cpp	15
studentai.h	22
versija_v2.0.cpp	24

Chapter 4

Class Documentation

4.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** ()=default
Final median result.
- **Studentas** (const **Studentas** &other)
Destructor.
- **Studentas** & **operator=** (const **Studentas** &other)
Copy constructor.
- **Studentas** (**Studentas** &&other) noexcept
Copy assignment.
- **Studentas** & **operator=** (**Studentas** &&other) noexcept
Move constructor.
- void **info** () const override
Move assignment.
- vector< int > & **refPaz** ()
Print student info.
- int & **refEgz** ()
Reference to grade vector.
- double & **refRez** ()
Reference to exam grade.
- double & **refMedrez** ()
Reference to final average.
- void **clearPaz** ()
Reference to final median.

- double [getRez](#) () const
Clear homework grades.
- double [getMedrez](#) () const
Get final average.
- int [getEgz](#) () const
Get final median.
- vector< int > & [getPaz](#) ()
Get exam grade.
- void [addPaz](#) (int p)
Get grade vector.
- void [setEgz](#) (int e)
Add homework grade.
- void [skaiciuoti](#) ()
Set exam grade.

Public Member Functions inherited from [Zmogus](#)

- string & [refVardas](#) ()
- string & [refPavarde](#) ()
Reference to first name.
- string [getVardas](#) () const
Reference to last name.
- string [getPavarde](#) () const
Get first name.
- void [setVardas](#) (const string &v)
Get last name.
- void [setPavarde](#) (const string &p)
Set first name.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [vardas](#)
- string [pavarde](#)

4.1.1 Detailed Description

Definition at line 43 of file [studentai.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Studentas() [1/2]

```
Studentas::Studentas (
    const Studentas & other) [inline]
```

Destructor.

Definition at line 56 of file [studentai.h](#).

4.1.2.2 Studentas() [2/2]

```
Studentas::Studentas (  
    Studentas && other) [inline], [noexcept]
```

Copy assignment.

Definition at line 75 of file [studentai.h](#).

4.1.3 Member Function Documentation

4.1.3.1 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

Get grade vector.

Definition at line 117 of file [studentai.h](#).

4.1.3.2 clearPaz()

```
void Studentas::clearPaz () [inline]
```

Reference to final median.

Definition at line 110 of file [studentai.h](#).

4.1.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Get final median.

Definition at line 113 of file [studentai.h](#).

4.1.3.4 getMedrez()

```
double Studentas::getMedrez () const [inline]
```

Get final average.

Definition at line 112 of file [studentai.h](#).

4.1.3.5 getPaz()

```
vector< int > & Studentas::getPaz () [inline]
```

Get exam grade.

Definition at line 115 of file [studentai.h](#).

4.1.3.6 getRez()

```
double Studentas::getRez () const [inline]
```

Clear homework grades.

Definition at line 111 of file [studentai.h](#).

4.1.3.7 info()

```
void Studentas::info () const [inline], [override], [virtual]
```

Move assignment.

kad veiktu ne abstrakciai

Implements [Zmogus](#).

Definition at line 102 of file [studentai.h](#).

4.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other) [inline]
```

Copy constructor.

Definition at line 64 of file [studentai.h](#).

4.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [inline], [noexcept]
```

Move constructor.

Definition at line 87 of file [studentai.h](#).

4.1.3.10 refEgz()

```
int & Studentas::refEgz () [inline]
```

Reference to grade vector.

Definition at line 107 of file [studentai.h](#).

4.1.3.11 refMedrez()

```
double & Studentas::refMedrez () [inline]
```

Reference to final average.

Definition at line 109 of file [studentai.h](#).

4.1.3.12 refPaz()

```
vector< int > & Studentas::refPaz () [inline]
```

Print student info.

Definition at line 106 of file [studentai.h](#).

4.1.3.13 refRez()

```
double & Studentas::refRez () [inline]
```

Reference to exam grade.

Definition at line 108 of file [studentai.h](#).

4.1.3.14 setEgz()

```
void Studentas::setEgz (  
    int e) [inline]
```

Add homework grade.

Definition at line 118 of file [studentai.h](#).

4.1.3.15 skaiciuoti()

```
void Studentas::skaiciuoti ()
```

Set exam grade.

Compute final average and median-based results

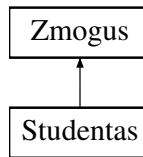
Definition at line 17 of file [funk.cpp](#).

The documentation for this class was generated from the following files:

- [studentai.h](#)
- [funk.cpp](#)

4.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- virtual void [info](#) () const =0
- string & [refVardas](#) ()
- string & [refPavarde](#) ()
Reference to first name.
- string [getVardas](#) () const
Reference to last name.
- string [getPavarde](#) () const
Get first name.
- void [setVardas](#) (const string &v)
Get last name.
- void [setPavarde](#) (const string &p)
Set first name.

Protected Attributes

- string [vardas](#)
- string [pavarde](#)

4.2.1 Detailed Description

Definition at line 24 of file [studentai.h](#).

4.2.2 Member Function Documentation

4.2.2.1 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Get first name.

Definition at line 37 of file [studentai.h](#).

4.2.2.2 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Reference to last name.

Definition at line 36 of file [studentai.h](#).

4.2.2.3 info()

```
virtual void Zmogus::info () const [pure virtual]
```

Implemented in [Studentas](#).

4.2.2.4 refPavarde()

```
string & Zmogus::refPavarde () [inline]
```

Reference to first name.

Definition at line 35 of file [studentai.h](#).

4.2.2.5 refVardas()

```
string & Zmogus::refVardas () [inline]
```

Definition at line 34 of file [studentai.h](#).

4.2.2.6 setPavarde()

```
void Zmogus::setPavarde (  
    const string & p) [inline]
```

Set first name.

Definition at line 39 of file [studentai.h](#).

4.2.2.7 setVardas()

```
void Zmogus::setVardas (  
    const string & v) [inline]
```

Get last name.

Definition at line 38 of file [studentai.h](#).

4.2.3 Member Data Documentation

4.2.3.1 pavarde

```
string Zmogus::pavarde [protected]
```

Definition at line 27 of file [studentai.h](#).

4.2.3.2 vardas

```
string Zmogus::vardas [protected]
```

Definition at line 26 of file [studentai.h](#).

The documentation for this class was generated from the following file:

- [studentai.h](#)

Chapter 5

File Documentation

5.1 funk.cpp

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 void Studentas::skaiciuoti()
00018 {
00020     if (paz.empty()) return;
00021
00022     int sum = 0;
00023     for (int x : paz) sum += x;
00024
00025     rez = 0.4 * (sum * 1.0 / paz.size()) + 0.6 * egz;
00026
00027     vector<int> temp = paz;
00028     sort(temp.begin(), temp.end());
00029
00030     if (temp.size() % 2 == 0)
00031         medrez = (temp[temp.size()/2 - 1] + temp[temp.size()/2]) / 2.0;
00032     else
00033         medrez = temp[temp.size()/2];
00034
00035     medrez = medrez * 0.4 + egz * 0.6;
00036 }
00037
00038 void inputas(vector<Studentas> &grupe)
00039 {
00040     Studentas A;
00041
00042     while (true)
00043     {
00044         int sum = 0;
00045
00046         cout << "Vardas ir pavarde: ";
00047         cin >> A.refVardas() >> A.refPavarde();
00048
00049         int i = 0;
00050         string ats;
00051         while (true)
00052         {
00053             int temp;
00054             cout << "Iveskite pazymi numeris " << i + 1 << ": ";
00055             try
00056             {
00057                 cin >> temp;
00058                 if (cin.fail())
```

```

00059         throw runtime_error("Bloga investis.");
00060     }
00061     catch (exception& e)
00062     {
00063         cin.clear();
00064         cin.ignore(10000, '\n');
00065         cout << e.what() << " Bandykite dar karta.\n";
00066         continue;
00067     }
00068
00069     A.refPaz().push_back(temp);
00070     sum += temp;
00071
00072     cout << "Ar toliau vesite pazymius? (y/n): ";
00073     cin >> ats;
00074     if (ats == "n") break;
00075     else if (ats == "y") cout << "Veskite toliau: ";
00076
00077     i++;
00078 }
00079
00080 cout << "Iveskite egzamino rezultata: ";
00081 try
00082 {
00083     cin >> A.refEgz();
00084     if (cin.fail())
00085         throw runtime_error("Bloga investis.");
00086 }
00087 catch (exception& e)
00088 {
00089     cin.clear();
00090     cin.ignore(10000, '\n');
00091     cout << e.what() << " Bandykite dar karta.\n";
00092     continue;
00093 }
00094
00095 if (!A.refPaz().empty())
00096 {
00097     A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00098
00099     sort(A.refPaz().begin(), A.refPaz().end());
00100
00101     int n = A.refPaz().size();
00102     if (n % 2 == 0)
00103         A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00104     else
00105         A.refMedrez() = A.refPaz()[n/2];
00106
00107     A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00108     A.skaiciuoti();
00109 }
00110
00111 grupe.push_back(A);
00112 A.clearPaz();
00113
00114 cout << "Ar toliau vesite mokinius? (y/n): ";
00115 cin >> ats;
00116 if (ats == "n") break;
00117 }
00118 }
00119
00120 void outputas(const vector<Studentas> &grupe)
00121 {
00122     cout << "-----" << endl;
00123     cout << left << setw(20) << "Vardas"
00124         << setw(20) << "Pavarde"
00125         << setw(20) << "Rezultatas (Vid.) /"
00126         << setw(20) << " Rezultatas (Med.)"
00127         << endl;
00128     cout << "-----" << endl;
00129
00130     for (const auto &A : grupe)
00131     {
00132         cout << left << setw(20) << A.getVardas()
00133             << setw(20) << A.getPavarde()
00134             << right << setw(20) << fixed << setprecision(2) << A.getRez()
00135             << setw(20) << A.getMedrez()
00136             << endl;
00137     }
00138 }
00139
00140 void failoutputas(const vector<Studentas> &grupe, string writing)
00141 {
00142     ofstream out(writing);
00143     out << "----- \n";
00144     out << left << setw(20) << "Vardas"
00145         << setw(20) << "Pavarde"

```

```

00148         « setw(20) « "Rezultatas (Vid.) /"
00149         « setw(20) « " Rezultatas (Med.)"
00150         « "\n";
00151     out « "----- \n";
00152
00153     for (const Studentas& s : grupe)
00154     {
00155         out « left « setw(20) « s.getVardas()
00156             « setw(20) « s.getPavarde()
00157             « fixed « setprecision(2)
00158             « setw(20) « s.getRez()
00159             « setw(20) « s.getMedrez()
00160             « '\n';
00161     }
00162
00163     cout « "Studentu skaicius: " « grupe.size() « endl;
00164 }
00165
00166 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus)
00167 {
00168     Studentas A;
00169     string lytis;
00170     string ats;
00171
00172     if (ArGeneruotiVardus)
00173     {
00174         int zmonsk = rand() % 100 + 1;
00175
00176         for(int iii = 0; iii < zmonsk; iii++)
00177         {
00178             int sum = 0;
00179
00180             lytis = lytgen();
00181             A.refVardas() = randomvardas(lytis);
00182             A.refPavarde() = randompavarde(lytis);
00183
00184             int pazsk = rand() % 100 + 1;
00185
00186             for (int ii = 0; ii < pazsk; ii++)
00187             {
00188                 int temp = rand() % 10 + 1;
00189                 A.refPaz().push_back(temp);
00190                 sum += temp;
00191             }
00192
00193             A.refEgz() = rand() % 10 + 1;
00194
00195             if (!A.refPaz().empty())
00196             {
00197                 A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00198
00199                 sort(A.refPaz().begin(), A.refPaz().end());
00200
00201                 int n = A.refPaz().size();
00202                 if (n % 2 == 0)
00203                     A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00204                 else
00205                     A.refMedrez() = A.refPaz()[n/2];
00206
00207                 A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00208             }
00209
00210             grupe.push_back(A);
00211             A.clearPaz();
00212         }
00213     }
00214     else
00215     {
00216         while (true)
00217         {
00218             int sum = 0;
00219
00220             cout « "Vardas ir pavarde: ";
00221             cin » A.refVardas() » A.refPavarde();
00222
00223             int pazsk = rand() % 100 + 1;
00224
00225             for (int i = 0; i < pazsk; i++)
00226             {
00227                 int temp = rand() % 10 + 1;
00228                 A.refPaz().push_back(temp);
00229                 sum += temp;
00230             }
00231
00232             A.refEgz() = rand() % 10 + 1;
00233
00234             if (!A.refPaz().empty())

```

```

00235         {
00236             A.refRez() = (sum * 1.0 / A.refPaz().size()) * 0.4 + A.refEgz() * 0.6;
00237
00238             sort(A.refPaz().begin(), A.refPaz().end());
00239
00240             int n = A.refPaz().size();
00241             if (n % 2 == 0)
00242                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00243             else
00244                 A.refMedrez() = A.refPaz()[n/2];
00245
00246             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00247         }
00248
00249         grupe.push_back(A);
00250         A.clearPaz();
00251
00252         cout << "Ar toliau vesite zmones? (y/n): ";
00253         cin >> ats;
00254         if (ats == "n") break;
00255     }
00256 }
00257 }
00258
00259 void failinputas(vector<Studentas> &grupe, string reading)
00260 {
00261     ifstream file(reading);
00262     grupe.reserve(1000000);
00263
00264     if (!file)
00265     {
00266         cout << "Nepavyko atidaryti failo\n";
00267         return;
00268     }
00269
00270     string line;
00271     getline(file, line);
00272
00273     while (getline(file, line))
00274     {
00275         istringstream iss(line);
00276
00277         Studentas A;
00278
00279         iss >> A.refVardas() >> A.refPavarde();
00280
00281         int pazymys;
00282
00283         while (iss >> pazymys)
00284             A.refPaz().push_back(pazymys);
00285
00286         if (!A.refPaz().empty())
00287         {
00288             A.refEgz() = A.refPaz().back();
00289             A.refPaz().pop_back();
00290         }
00291
00292         if (!A.refPaz().empty())
00293         {
00294             int sum = 0;
00295             for (int x : A.refPaz())
00296                 sum += x;
00297
00298             sort(A.refPaz().begin(), A.refPaz().end());
00299
00300             int n = A.refPaz().size();
00301
00302             if (n % 2 == 0)
00303                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00304             else
00305                 A.refMedrez() = A.refPaz()[n/2];
00306
00307             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00308             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00309         }
00310
00311         grupe.push_back(A);
00312     }
00313 }
00314
00315 string randomvardas(string lytis)
00316 {
00317     if (lytis == "mot")
00318         return fvardai[rand() % fvardai.size()];
00319     else
00320         return mvardai[rand() % mvardai.size()];
00321 }

```

```

00322 }
00323
00324 string randompavarde(string lytis)
00325 {
00326     int index = rand() % pavardes.size();
00327     string galune;
00328
00329     if (lytis == "mot")
00330         galune = fgalunes[rand() % fgalunes.size()];
00331     else
00332         galune = mgalunes[rand() % mgalunes.size()];
00333
00334     return pavardes[index] + galune;
00335 }
00336
00337 string lytgen()
00338 {
00339     return rand() % 2 == 0 ? "vyr" : "mot";
00340 }
00341
00342
00343 bool cmpVardas(const Studentas &a, const Studentas &b)
00344 {
00345     return a.getVardas() < b.getVardas();
00346 }
00347
00348 bool cmpPavarde(const Studentas &a, const Studentas &b)
00349 {
00350     return a.getPavarde() < b.getPavarde();
00351 }
00352
00353 bool cmpRez(const Studentas &a, const Studentas &b)
00354 {
00355     return a.getRez() < b.getRez();
00356 }
00357
00358 bool cmpMedrez(const Studentas &a, const Studentas &b)
00359 {
00360     return a.getMedrez() < b.getMedrez();
00361 }
00362
00363 bool cmpEgz(const Studentas &a, const Studentas &b)
00364 {
00365     return a.getEgz() < b.getEgz();
00366 }
00367
00368
00369 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas)
00370 {
00371     if (pasirinkimas == 1)
00372         sort(grupe.begin(), grupe.end(), cmpVardas);
00373     else if (pasirinkimas == 2)
00374         sort(grupe.begin(), grupe.end(), cmpPavarde);
00375     else if (pasirinkimas == 3)
00376         sort(grupe.begin(), grupe.end(), cmpRez);
00377     else if (pasirinkimas == 4)
00378         sort(grupe.begin(), grupe.end(), cmpMedrez);
00379     else if (pasirinkimas == 5)
00380         sort(grupe.begin(), grupe.end(), cmpEgz);
00381 }
00382
00383
00384 void failgeneravimas()
00385 {
00386     int sk = 1000;
00387     for (int it = 0; it < 5; it++)
00388     {
00389         cout << "pradedamas " << sk << " eiluciu failo generavimo laiko skaiciavimas" << endl;
00390         auto input_start = chrono::high_resolution_clock::now();
00391
00392         ofstream gen("gen" + to_string(sk) + ".txt");
00393         gen << left << setw(20) << "STUD. VARDAS"
00394             << setw(20) << "STUD. PAVARDE"
00395             << setw(20) << "STUD PAZYMAI (PASKUTINIS EGZAMINO)\n";
00396
00397         for (int genit = 1; genit <= sk; genit++)
00398         {
00399             gen << left << setw(20) << "genVardas" + to_string(genit)
00400                 << setw(20) << "genPavarde" + to_string(genit);
00401
00402             for (int i = 1; i <= 16; i++)
00403                 gen << setw(20) << rand() % 10 + 1;
00404
00405             gen << "\n";
00406         }
00407
00408         gen.close();

```

```

00409         sk *= 10;
00410
00411         auto input_end = chrono::high_resolution_clock::now();
00412         chrono::duration<double> input_diff = input_end - input_start;
00413         cout << "Failu sukurimas + uzdarymas uztruko: " << input_diff.count() << " s\n";
00414     }
00415 }
00416
00417 void genirasydas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00418 string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija)
00419 {
00420     int sk = 1000;
00421
00422     for (int it = 0; it < 5; it++)
00423     {
00424         string filename = "gen" + to_string(sk) + ".txt";
00425
00426         cout << "Pradedamas failo nuskaitymas: " << filename << endl;
00427         auto start_read = chrono::high_resolution_clock::now();
00428
00429         ifstream file(filename);
00430
00431         if (!file)
00432         {
00433             cout << "Nepavyko atidaryti failo\n";
00434             sk *= 10;
00435             continue;
00436         }
00437
00438         string line;
00439
00440         getline(file, line); // skip header
00441
00442         while (getline(file, line))
00443         {
00444             istringstream iss(line);
00445
00446             Studentas A;
00447
00448             iss >> A.refVardas() >> A.refPavarde();
00449
00450             int paz;
00451             while (iss >> paz)
00452                 A.refPaz().push_back(paz);
00453
00454             if (A.refPaz().empty())
00455                 continue;
00456
00457             A.refEgz() = A.refPaz().back();
00458             A.refPaz().pop_back();
00459
00460             int sum = 0;
00461             for (int x : A.refPaz())
00462                 sum += x;
00463
00464             A.refRez() = 0.4 * (sum * 1.0 / A.refPaz().size()) + 0.6 * A.refEgz();
00465
00466             sort(A.refPaz().begin(), A.refPaz().end());
00467
00468             int n = A.refPaz().size();
00469
00470             if (n % 2 == 0)
00471                 A.refMedrez() = (A.refPaz()[n/2 - 1] + A.refPaz()[n/2]) / 2.0;
00472             else
00473                 A.refMedrez() = A.refPaz()[n/2];
00474
00475             A.refMedrez() = A.refMedrez() * 0.4 + A.refEgz() * 0.6;
00476
00477             grupė.push_back(A);
00478         }
00479
00480         file.close();
00481
00482         auto end_read = chrono::high_resolution_clock::now();
00483         chrono::duration<double> diff_read = end_read - start_read;
00484
00485         cout << "Failo nuskaitymas uztruko: " << diff_read.count() << " s\n";
00486
00487         auto start_rikiavimas = chrono::high_resolution_clock::now();
00488         rikiuoti(grupe, rik_pasirinkimas);
00489         auto end_rikiavimas = chrono::high_resolution_clock::now();
00490         chrono::duration<double> diff_rikiavimas = end_rikiavimas - start_rikiavimas;
00491
00492         cout << "Rikiavimas failui " << filename << " uztruko: " << diff_rikiavimas.count() << " s\n";
00493
00494         auto start_skirstymas = chrono::high_resolution_clock::now();

```

```

00496
00497     if (strategija == SkirstymoStrategija::PIRMAS)
00498         skirstyti(grupe, protingi, neprotingi);
00499     else if (strategija == SkirstymoStrategija::ANTRAS)
00500         skirstyti2(grupe, neprotingi);
00501     else if (strategija == SkirstymoStrategija::TRECIAS)
00502         skirstyti3(grupe, neprotingi);
00503
00504     auto end_skirstymas = chrono::high_resolution_clock::now();
00505     chrono::duration<double> diff_skirstymas = end_skirstymas - start_skirstymas;
00506
00507     cout << "Protingu/Neprotingu skirstymas failui " << filename << " uztruks: " <<
diff_skirstymas.count() << " s\n";
00508
00509     if (strategija == SkirstymoStrategija::PIRMAS)
00510     {
00511         cout << "Protingu ";
00512         failoutputas(protingi, "kursiokai_geri_" + to_string(sk) + ".txt");
00513         cout << "Neprotingu ";
00514         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00515     }
00516     else
00517     {
00518         cout << "Protingu ";
00519         failoutputas(grupe, "kursiokai_geri_" + to_string(sk) + ".txt");
00520         cout << "Neprotingu ";
00521         failoutputas(neprotingi, "kursiokai_blogi_" + to_string(sk) + ".txt");
00522     }
00523
00524     cout << "\n";
00525
00526     grupe.clear();
00527     protingi.clear();
00528     neprotingi.clear();
00529
00530     logResults(
00531         "vector",
00532         sk,
00533         diff_read.count(),
00534         diff_rikiavimas.count(),
00535         diff_skirstymas.count()
00536     );
00537
00538     sk *= 10;
00539 }
00540 }
00541
00542
00543 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
neprotingi)
00544 {
00545     for (const auto& s : grupe)
00546     {
00547         if (s.getRez() >= 5.0)
00548             protingi.push_back(s);
00549         else
00550             neprotingi.push_back(s);
00551     }
00552 }
00553
00554
00555 void logResults(const string& container, int size,
00556                 double read_t, double sort_t, double split_t)
00557 {
00558     ofstream out("results.csv", ios::app);
00559     out << container << ", "
00560         << size << ", "
00561         << read_t << ", "
00562         << sort_t << ", "
00563         << split_t << "\n";
00564 }
00565
00566
00567 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00568 {
00569     auto it = partition(grupe.begin(), grupe.end(),
00570         [](const Studentas& s) { return s.getRez() >= 5.0; });
00571     neprotingi.assign(it, grupe.end());
00572     grupe.erase(it, grupe.end());
00573 }
00574
00575
00576
00577 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi)
00578 {
00579     neprotingi.reserve(grupe.size());
00580

```

```

00581     auto it = remove_if(grupe.begin(), grupe.end(),
00582         [&](const Studentas& s)
00583         {
00584             if (s.getRez() < 5.0)
00585             {
00586                 neprotingi.push_back(s);
00587                 return true;
00588             }
00589             return false;
00590         });
00591     grupe.erase(it, grupe.end());
00592 }
00593
00594 ostream& operator<<(ostream& os, const Studentas& s)
00595 {
00596     os << left << setw(20) << s.getVardas()
00597         << setw(20) << s.getPavarde()
00598         << setw(10) << fixed << setprecision(2) << s.getRez()
00599         << setw(10) << s.getMedrez();
00600     return os;
00601 }
00602
00603 istream& operator>>(istream& is, Studentas& s)
00604 {
00605     s.refPaz().clear();
00606     is >> s.refVardas() >> s.refPavarde();
00607     int paz;
00608     while (is >> paz) {
00609         s.refPaz().push_back(paz);
00610     }
00611     if (!s.refPaz().empty()) {
00612         s.refEgz() = s.refPaz().back();
00613         s.refPaz().pop_back();
00614         s.skaiciuoti();
00615     }
00616     return is;
00617 }
00618
00619
00620
00621
00622

```

5.2 studentai.h

```

00001 #ifndef STUDENTAI_H
00002 #define STUDENTAI_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <algorithm>
00007
00008 using namespace std;
00009
00010 const vector<string> mvardai = {"Vytenis", "Tomas", "Jonas", "Matas", "Simas", "Mantas", "Arnas"};
00011 const vector<string> fvardai = {"Egle", "Viktorija", "Vakare", "Inga", "Ema", "Marija", "Janina"};
00012 const vector<string> pavardes = {"Macerausk", "Jankausk", "Kazlausk", "Svilpausk", "Drugeliausk",
00013     "Briedausk"};
00014 const vector<string> mgalunes = {"as", "aitis"};
00015 const vector<string> fgalunes = {"iene", "aite", "yte"};
00016
00017 enum class SkirstymoStrategija
00018 {
00019     PIRMAS,
00020     ANTRAS,
00021     TRECIAS
00022 };
00023
00024 class Zmogus {
00025 protected:
00026     string vardas;
00027     string pavarde;
00028
00029 public:
00030     virtual ~Zmogus() = default;
00031
00032     virtual void info() const = 0;
00033
00034     string& refVardas() { return vardas; }
00035     string& refPavarde() { return pavarde; }
00036     string getVardas() const { return vardas; }
00037     string getPavarde() const { return pavarde; }
00038     void setVardas(const string& v) { vardas = v; }

```

```

00039     void setPavarde(const string& p) { pavarde = p; }
00040 };
00041
00042
00043 class Studentas : public Zmogus {
00044 private:
00045     vector<int> paz;
00046     int egz = 0;
00047     double rez = 0.0;
00048     double medrez = 0.0;
00049
00050 public:
00051     Studentas() = default;
00052
00053     // Rule of Five
00054     ~Studentas() = default;
00055
00056     Studentas(const Studentas& other)
00057         : Zmogus(other),
00058           paz(other.paz),
00059           egz(other.egz),
00060           rez(other.rez),
00061           medrez(other.medrez)
00062     {}
00063
00064     Studentas& operator=(const Studentas& other) {
00065         if (this != &other) {
00066             Zmogus::operator=(other);
00067             paz = other.paz;
00068             egz = other.egz;
00069             rez = other.rez;
00070             medrez = other.medrez;
00071         }
00072         return *this;
00073     }
00074
00075     Studentas(Studentas&& other) noexcept
00076         : Zmogus(std::move(other)),
00077           paz(std::move(other.paz)),
00078           egz(other.egz),
00079           rez(other.rez),
00080           medrez(other.medrez)
00081     {
00082         other.egz = 0;
00083         other.rez = 0;
00084         other.medrez = 0;
00085     }
00086
00087     Studentas& operator=(Studentas&& other) noexcept {
00088         if (this != &other) {
00089             Zmogus::operator=(std::move(other));
00090             paz = std::move(other.paz);
00091             egz = other.egz;
00092             rez = other.rez;
00093             medrez = other.medrez;
00094
00095             other.egz = 0;
00096             other.rez = 0;
00097             other.medrez = 0;
00098         }
00099         return *this;
00100     }
00101
00102     void info() const override {
00103         cout << "Studentas: " << vardas << " " << pavarde << endl;
00104     }
00105
00106     vector<int>& refPaz() { return paz; }
00107     int& refEgz() { return egz; }
00108     double& refRez() { return rez; }
00109     double& refMedrez() { return medrez; }
00110     void clearPaz() { paz.clear(); }
00111     double getRez() const { return rez; }
00112     double getMedrez() const { return medrez; }
00113     int getEgz() const { return egz; }
00114
00115     vector<int>& getPaz() { return paz; }
00116
00117     void addPaz(int p) { paz.push_back(p); }
00118     void setEgz(int e) { egz = e; }
00119
00120     void skaiciuoti();
00121 };
00122
00124 const string irasyimo_failas = "kursiokai.txt";
00125 const string irasyimo_failas_geras = "kursiokai_geri.txt";
00126 const string irasyimo_failas_blogas = "kursiokai_blogi.txt";

```

```

00127
00128 void inputas(vector<Studentas> &grupe);
00129 void failinputas(vector<Studentas> &grupe, string reading);
00130 void randominputas(vector<Studentas> &grupe, bool ArGeneruotiVardus);
00131 void outputas(const vector<Studentas> &grupe);
00132 void failoutputas(const vector<Studentas> &grupe, string writing);
00133 void failgeneravimas();
00134 void genirasymas(vector<Studentas> &grupe, vector<Studentas> &protingi, vector<Studentas> &neprotingi,
00135     string writing_good, string writing_bad, int rik_pasirinkimas, SkirstymoStrategija strategija);
00136 string lytgen();
00137 string randomvardas(string lytis);
00138 string randompavarde(string lytis);
00139 void rikiuoti(vector<Studentas> &grupe, int pasirinkimas);
00140 bool cmpVardas(const Studentas &a, const Studentas &b);
00141 bool cmpPavarde(const Studentas &a, const Studentas &b);
00142 bool cmpRez(const Studentas &a, const Studentas &b);
00143 bool cmpMedrez(const Studentas &a, const Studentas &b);
00144 bool cmpEgz(const Studentas &a, const Studentas &b);
00145 void skirstyti(const vector<Studentas>& grupe, vector<Studentas>& protingi, vector<Studentas>&
    neprotingi);
00146 void logResults(const string& container, int size,
00147     double read_t, double sort_t, double split_t);
00148 void skirstyti2(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00149 void skirstyti3(vector<Studentas>& grupe, vector<Studentas>& neprotingi);
00150
00151 ostream& operator<<(ostream& os, const Studentas& s);
00152 istream& operator>>(istream& is, Studentas& s);
00153
00154 #endif

```

5.3 versija_v2.0.cpp

```

00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <string>
00005 #include <cmath>
00006 #include <algorithm>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <chrono>
00012 #include "studentai.h"
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 int main()
00018 {
00019     srand(time(NULL));
00020
00021     vector<Studentas> grupe;
00022     vector<Studentas> protingi;
00023     vector<Studentas> neprotingi;
00024
00025     grupe.reserve(10000000);
00026
00027     string genpas;
00028     cout << "Norite generuoti 5 failus? y/n " << endl;
00029     cin >> genpas;
00030     if (genpas == "y") failgeneravimas();
00031
00032     cout << "--- Zmogus kurimo testavimas ---" << endl;
00033
00034     // Zmogus z;
00035
00036     cout << "--- Rule of Five dalykai ---" << endl;
00037
00038     Studentas a;
00039     a.setVardas("Jonas");
00040     a.setPavarde("Jonaitis");
00041     a.addPaz(10);
00042     a.addPaz(9);
00043     a.setEgz(8);
00044     a.skaiciuoti();
00045
00046     cout << setw(20) << "Original: " << a << endl;
00047
00048     Studentas b = a; // copy constructor
00049     cout << setw(20) << "Copy ctor: " << b << endl;
00050
00051     Studentas c;

```

```

00052     c = a;                                // copy assignment
00053     cout << setw(20) << "Copy assign: " << c << endl;
00054
00055     Studentas d = move(a); // move constructor
00056     cout << setw(20) << "Move ctor: " << d << endl;
00057
00058     Studentas e;
00059     e = move(b); // move assignment
00060     cout << setw(20) << "Move assign: " << e << endl;
00061
00062     cout << "=== End of Test ===" << endl << endl;
00063
00064
00065     int pasirinkimas = 4;
00066     string isvedimotipas, skaitymo_pasirinkimas, skaitymo_failas;
00067     cout << "Kokios norite ivesties? (1 - ranka arba is failo, 2 - generuoti tik pazymius, " << endl
00068     << " 3 - generuoti studentu vardus, pavardes ir pazymius, 4 - skaityti sugeneruotus failus; 5 arba
00069     kitas simbolis - baigti darba)" << endl;
00070     cin >> pasirinkimas;
00071
00072     int rikiavimo_pasirinkimas;
00073     cout << "Pagal ka rikiuoti?" << endl
00074     << "1 - vardas" << endl
00075     << "2 - pavarde" << endl
00076     << "3 - galutinis (vidurkis)" << endl
00077     << "4 - galutinis (mediana)" << endl
00078     << "5 - egzaminas" << endl;
00079     cin >> rikiavimo_pasirinkimas;
00080
00081     auto input_start = chrono::high_resolution_clock::now();
00082
00083     if (pasirinkimas == 1)
00084     {
00085         cout << "Ar skaityti duomenis is failo? (y - is failo / n - ranka)";
00086         cin >> skaitymo_pasirinkimas;
00087         if (skaitymo_pasirinkimas == "y")
00088         {
00089             string skaitymo_failas;
00090             cout << "Iveskite failo pavadinima: ";
00091             cin >> skaitymo_failas;
00092             failinputas(grupe, skaitymo_failas);
00093         }
00094         else inputas(grupe);
00095     }
00096     else if (pasirinkimas == 2)
00097     {
00098         randominputas(grupe, false);
00099     }
00100     else if (pasirinkimas == 3)
00101     {
00102         randominputas(grupe, true);
00103     }
00104     else if (pasirinkimas == 4)
00105     {
00106         genirasymas(grupe, protingi, neprotingi, irasyimo_failas_geras, irasyimo_failas_blogas,
00107         rikiavimo_pasirinkimas, SkirstymoStrategija::TRECIIAS);
00108         return 0;
00109     }
00110     else
00111     {
00112         cout << "darbas baigtas" << endl;
00113         return 0;
00114     }
00115
00116     auto input_end = chrono::high_resolution_clock::now();
00117     chrono::duration<double> input_diff = input_end - input_start;
00118     cout << "Duomeniu nuskaitymas uztruks: " << input_diff.count() << " s \n";
00119
00120     cout << "Ar rasyti i faila? (y/n)" << endl;
00121     cin >> isvedimotipas;
00122
00123     auto start = std::chrono::high_resolution_clock::now(); // Paleisti
00124
00125     rikiuoti(grupe, rikiavimo_pasirinkimas);
00126     //skirstyti(grupe, protingi, neprotingi)
00127     //skirstyti2(grupe, neprotingi);
00128     skirstyti3(grupe, neprotingi);
00129
00130     if (isvedimotipas == "y")
00131     {
00132         cout << "pradedu rasyt" << endl;
00133         failoutputas(protingi, "kursiokai_geri.txt");
00134         failoutputas(grupe, "kursiokai_geri_grupe.txt");
00135         failoutputas(neprotingi, "kursiokai_blogi.txt");
00136         cout << "Baigiu rasyt" << endl;
00137     }

```

```
00138     else
00139     {
00140         outputas(grupe);
00141     }
00142
00143     // is pavyzdzio
00144     auto end = chrono::high_resolution_clock::now(); // Stabdyti
00145     chrono::duration<double> diff = end-start; // Skirtumas (s)
00146     cout << "Programos rikiavimas ir isvedimas uztruko: " << diff.count() << " s\n";
00147
00148     cout << "program finished." << endl;
00149     return 0;
00150 }
```

Index

addPaz
 Studentas, [9](#)

clearPaz
 Studentas, [9](#)

getEgz
 Studentas, [9](#)

getMedrez
 Studentas, [9](#)

getPavarde
 Zmogus, [12](#)

getPaz
 Studentas, [9](#)

getRez
 Studentas, [9](#)

getVardas
 Zmogus, [12](#)

info
 Studentas, [10](#)
 Zmogus, [13](#)

operator=
 Studentas, [10](#)

pavarde
 Zmogus, [13](#)

refEgz
 Studentas, [10](#)

refMedrez
 Studentas, [10](#)

refPavarde
 Zmogus, [13](#)

refPaz
 Studentas, [11](#)

refRez
 Studentas, [11](#)

refVardas
 Zmogus, [13](#)

setEgz
 Studentas, [11](#)

setPavarde
 Zmogus, [13](#)

setVardas
 Zmogus, [13](#)

skaiciuoti
 Studentas, [11](#)

Studentas, [7](#)

addPaz, [9](#)

clearPaz, [9](#)

getEgz, [9](#)

getMedrez, [9](#)

getPaz, [9](#)

getRez, [9](#)

info, [10](#)

operator=, [10](#)

refEgz, [10](#)

refMedrez, [10](#)

refPaz, [11](#)

refRez, [11](#)

setEgz, [11](#)

skaiciuoti, [11](#)

Studentas, [8](#)

vardas
 Zmogus, [13](#)

Zmogus, [12](#)

 getPavarde, [12](#)

 getVardas, [12](#)

 info, [13](#)

 pavarde, [13](#)

 refPavarde, [13](#)

 refVardas, [13](#)

 setPavarde, [13](#)

 setVardas, [13](#)

 vardas, [13](#)